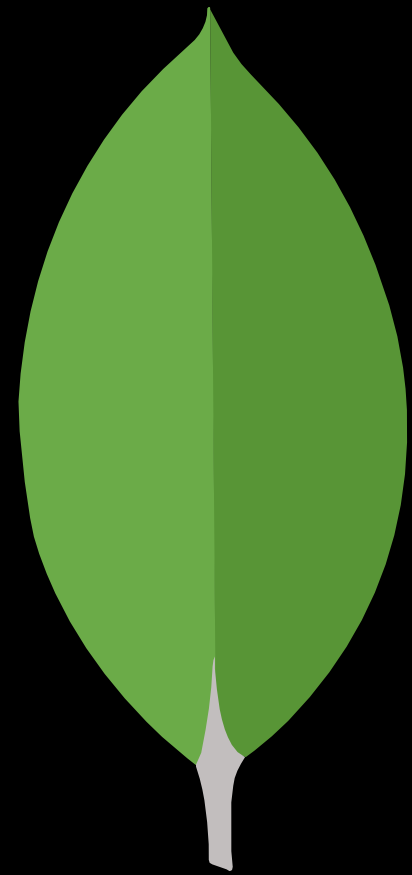




MONGO

DB

A Modern Database For
A modern Developers like
You

By – Mohammad Raza

Roadmap



Section 1: Introduction to MongoDB

What is Database, A brief History of Databases

SQL vs NoSQL

What is MongoDB and Why use it?

MongoDB Atlas (Cloud Database) setup

Setting up MongoDB Compass (GUI)



Connecting to MongoDB using Compass & Atlas



Roadmap

Section 2: MongoDB Basics (CRUD Operations)

Create a Database & Collection
Insert Documents (insertOne, insertMany)

Read Documents (find, findOne)

Update Documents (updateOne, updateMany, \$set, \$inc)

Delete Documents (deleteOne, deleteMany)

MongoDB Query Filters (Comparison & Logical Operators)

\$eq, \$ne, \$gt, \$lt, \$in, \$and, \$or, etc.

Roadmap



Section 3: Data Modeling

Documents, Collections, Databases — structure samjho

Data types in MongoDB

Embedded vs Referenced Documents

Designing Schema for Real-World Apps (e.g. Users & Orders)

Roadmap



Section 4: Advanced Querying

Sorting & Limiting Results

Projection (select specific fields)

Aggregation Pipeline Basics

\$match, \$group, \$sort, \$sum, \$avg, \$count

\$lookup for joins between collections

\$project for custom output



Roadmap



Section 5: Indexing & Performance

What are Indexes & Why use them

Creating Indexes (createIndex)

How Indexes improve query performance

WHAT IS DATABASE, HISTORY OF DATABASES, SQL VS NOSQL



1. What is a Database?

- **A Database is an organized collection of data that can be easily accessed, managed, and updated.**
- **In simple words, it's like a digital storage where we keep our data in a structured or unstructured form**

Example:

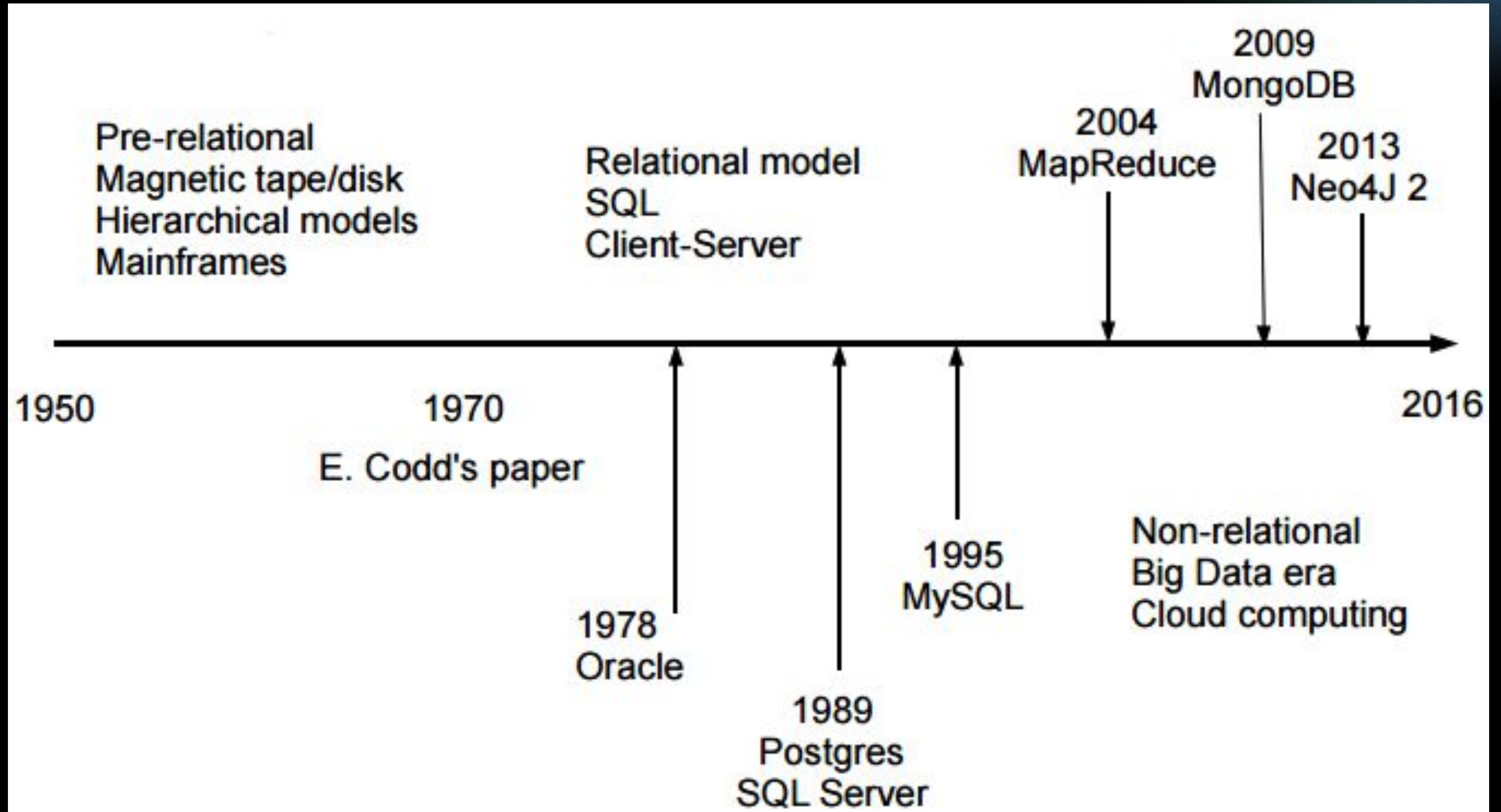
- **Think of your mobile contact list — names, phone numbers, and emails.**

That's a small database of contacts.

In real life:

- **YouTube stores video info (title, views, likes, comments , subscribers ... etc.)**
Amazon stores product details (name, price, stock, reviews etc...)

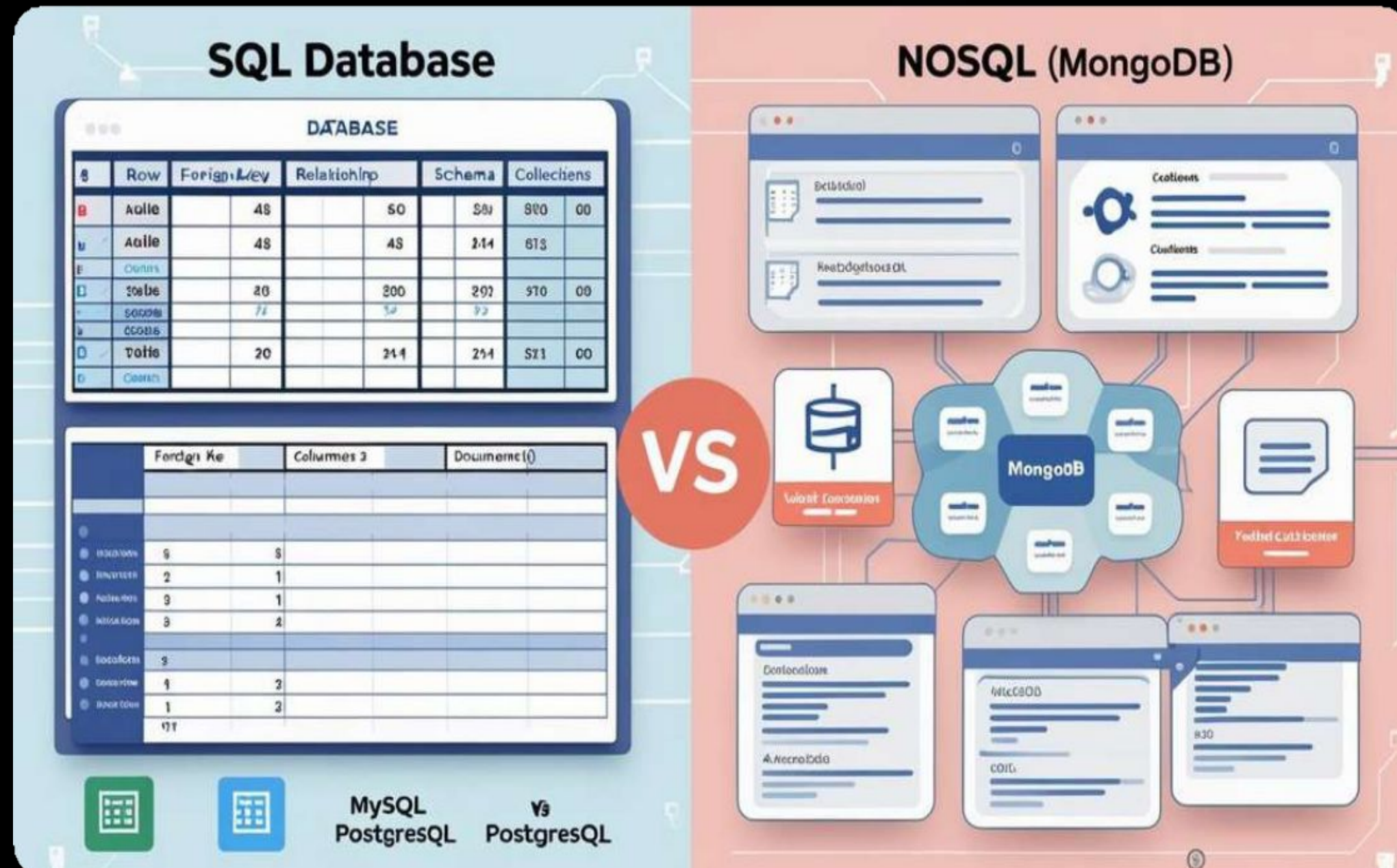
2. History of Databases :)



2. Types of Databases

Mainly there are two types:)

- **SQL (Relational Databases)**
- **NoSQL (Non-Relational Databases)**





SQL (Relational Database)

- SQL stands for Structured Query Language.

- These databases store data in tables — rows and columns — just like Excel.

- Each table is related to another table using relationships (like primary key & foreign key).

🧠 Examples: MySQL, PostgreSQL, Oracle,

SQL Server

Users			
Name	Age	Address	Skills
Alice	28	New York	SQL
Age	28	Address	Address
Bob	32	London	Excel
Charlie	24	Skills	Javascript
Charlie	24	Javascript	React
	24	Delhi	React



NoSQL (Non-Relational Database)

- NoSQL means Not Only SQL — used for unstructured or semi-structured data.
- Data is stored in forms like Documents, Key-Value pairs, Graphs, or Columns.
- 🧠 Examples: MongoDB, Firebase, Cassandra, Redis

✓ Best for:
Large-scale applications (like social media, IoT, real-time data)
Flexible structure (no fixed schema)
Fast performance and easy scalability

```
< {  
  _id: ObjectId("65849aae7de3dc96745d21d4"),  
  name: 'Raj',  
  age: 20,  
  rollno: 65,  
  marks: 90  
}
```

[Home](#)[About](#)[Content](#)[Others](#)

WHAT IS MONGODB

What is MongoDB?

- MongoDB is a NoSQL (Non-Relational) database.
- This means it doesn't store data in traditional tables with rows and columns like SQL databases.
- Instead, MongoDB stores data as documents in a format similar to JSON (JavaScript Object Notation).

```
< {  
  _id: ObjectId("65849aae7de3dc96745d21d4"),  
  name: 'Raj',  
  age: 20,  
  rollno: 65,  
  marks: 90  
}
```

How MongoDB Works

- Instead of tables, MongoDB uses collections.
- Each collection contains multiple documents.
- Each document can have a different structure (no fixed schema).
- Data is stored in BSON (Binary JSON) format — lightweight and fast for reading and writing.

```
< {  
  _id: ObjectId("65849aae7de3dc96745d21d4"),  
  name: 'Raj',  
  age: 20,  
  rollno: 65,  
  marks: 90  
}
```

Why Use MongoDB?

1. Flexible Schema:

- No fixed columns — every document can have different fields.
- Perfect for dynamic or frequently changing data.

2. Fast & Scalable:

- MongoDB supports horizontal scaling, which means you can add more servers to increase performance easily.

3. Easy for JavaScript Developers:

Since MongoDB uses a JSON-like format, it fits perfectly with JavaScript-based frameworks like React and Node.js.

Why Use MongoDB?

4. Great for Big Data & Real-Time Apps:

- **It handles large amounts of unstructured data efficiently — ideal for social media apps, chat apps, and analytics dashboards.**

5. Open Source & Cloud Ready:

MongoDB is free and open-source, and with MongoDB Atlas, you can deploy it easily on the cloud.

Where MongoDB is Used



Uber

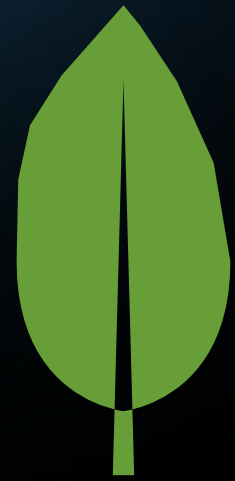
Adobe

Use Cases:

- Social media
- platforms
- E-commerce websites
- IoT applications

[Home](#)[About](#)[Content](#)[Others](#)

MONGODB ATLAS



MODERN WAY TO USE MONGODB

What is MongoDB Atlas?

- **MongoDB Atlas is a cloud-based database service provided by MongoDB.**
- **It allows you to store, manage, and access data online — without installing MongoDB on your local machine.**
- **It's a Database-as-a-Service (DBaaS) platform — meaning MongoDB handles all the infrastructure, backups, scaling, and security for you.**
-  **In simple words:**
- **Instead of running MongoDB on your laptop, you can create it on the cloud and connect to it from anywhere — using your app or MongoDB Compass.**

Why Use MongoDB Atlas?

✓ 1. No Installation Required:

You don't need to set up MongoDB manually — everything runs in the cloud.

✓ 2. Access from Anywhere:

You can connect from your laptop, a server, or even your mobile app — all through a connection URL.

✓ 3. High Availability:

Your data is automatically replicated across multiple servers to avoid downtime.

✓ 4. Auto Scaling:

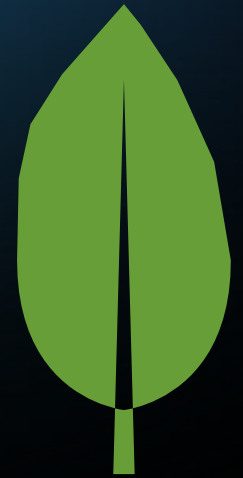
As your app grows, Atlas automatically adjusts your database resources (CPU, RAM, Storage).

✓ 5. Security:

Built-in encryption, authentication, and firewall rules make it highly secure.

✓ 6. Free Tier Available:

MongoDB Atlas offers a free M0 cluster, perfect for students, beginners, or small projects.

[Home](#)[About](#)[Content](#)[Others](#)

SETTING UP MONGODB COMPASS (GUI)

What is MongoDB Compass?

- **MongoDB Compass is an official GUI provided by MongoDB.**
- **It helps you view, insert, update, delete, and query data in your database visually.**
- **Think of it as a “MongoDB Dashboard” — like PhpMyAdmin for MySQL.**

Key Features

- **Connect to local or cloud MongoDB databases**
- **Browse databases and collections**
- **Run queries visually**
- **Analyze schema structure**
- **View performance stats**

Download MongoDB Compass

- Go to <https://www.mongodb.com/try/download/compass>
- Choose Windows as your operating system.
- Select .msi (Windows Installer) package.
- Click Download and wait for it to complete.

Install MongoDB Compass

- Once downloaded, open the installer.
- Click Next → Next → Finish (default options are fine).
- After installation, open MongoDB Compass from your Start Menu (Windows) or Applications (Mac).

[Home](#)[About](#)[Content](#)[Others](#)

CREATE DATABASE & COLLECTION



Basic MongoDB Commands



1. Show All Databases

show dbs



Displays all the databases available in your MongoDB server.



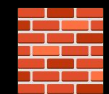
2. Create or Switch Databases

use myDatabase

If it doesn't exist, MongoDB creates it when you insert your first document.



Basic MongoDB Commands



3. Create a Collection

```
db.createCollection("users")
```



Creates a new collection named users.



4. Show All Collections

```
show collections
```



Lists all the collections inside the current database.

[Home](#)[About](#)[Content](#)[Others](#)

INSERT DOCUMENT

insertOne, insertMany

Insert One Document

```
db.users.insertOne( {  
  "name": "Raza",  
  "age": 22,  
  "city": "Pune"  
});
```

Insert Many Documents

```
db.users.insertMany([  
  { name: "Tushar", age: 21 },  
  { name: "Omkar", age: 23},  
  {name : "shraddha", age : 22}  
]);
```

[Home](#)[About](#)[Content](#)[Others](#)

READ DOCUMENTS

find, findOne

View All Documents in a Collection

db.users.find()

👉 Displays all the data stored in the users collection.

Pretty Print (Readable Output)

db.users.find().pretty()

👉 Shows the documents in a more readable (formatted) way.

Find One Document

```
db.users.findOne({ name: "Tushar" })
```

👉 This will return the first document where the name is “Danish”.

Find Without Condition

```
db.users.findOne()
```

👉 Returns the first document in the collection.



UPDATE DOCUMENTS

**updateOne, updateMany, \$set,
\$inc**



UPDATE Documents

In MongoDB, after inserting documents, we can modify existing records using update operations.

- **updateOne()**
- **updateMany()**
- **\$set operator**
- **\$inc operator**



updateOne()

- Used to update a single document that matches a filter condition.
- Only the first matching document will be updated, even if multiple documents match.

- `db.collection.updateOne( { filter } , { update } )`

```
>>> db.students.updateOne(  
    { name: "Omkar" },  
    { $set: { marks: 85 } }  
)
```



updateMany()

- Used to update multiple documents that match the filter condition. All
- matching documents will be updated simultaneously.

- `db.collection.updateMany( { filter } , { update } )`

```
>>> db.students.updateMany(  
    { age: 20 },  
    { $set: { marks: 75 } }  
)
```



\$set Operator

- Used to modify or add a new field to the document. If
- the field exists → updates it.
- If it doesn't exist → creates it.

```
>>> db.students.updateOne(  
    { name: "Zara" },  
    { $set: { grade: "A" } }  
)
```



\$inc Operator

- Used to increment or decrement a numeric field's value.
- Positive number → increases the value.
- Negative number → decreases the value

```
>>> db.students.updateOne(  
    { name: "shraddha" },  
    { $inc: { marks: -5 } }  
)
```

✓ shraddha's marks will decrease
by 5.

```
>>> db.students.updateMany(  
    {},  
    { $inc: { age: +1 } }  
)
```

✓ All students' ages will increase by 1.



Combining Operators

- You can combine \$set and \$inc together:

```
>>> db.students.updateOne(  
    { name: "sushant" },  
    {  
        $set: { grade: "B" },  
        $inc: { marks: 10 }  
    }  
)
```

Adds grade "B" and increases marks by 10 for sushant.



Check Update Results

- `updateOne()` and `updateMany()` return a result object:

```
>>> {
  acknowledged: true,
  matchedCount: 2,
  modifiedCount: 2
}
```

- `acknowledged` → true if the operation was successful.
- `matchedCount` → number of documents matched.
- `modifiedCount` → number of documents actually updated.

[Home](#)[About](#)[Content](#)[Others](#)

DELETE DOCUMENTS

(deleteOne, deleteMany)

DELETE Documents

After inserting and updating documents, sometimes we need to remove unnecessary or old data from our collection — and for that, we use delete operations.

- **deleteOne()**
- **deleteMany()**
- **How to verify deletion results.**



deleteOne()

- Used to delete a single document that matches a given filter condition. If
- multiple documents match, only the first one will be deleted.

- **db.collection.deleteOne({ name: "zara" })**

Explanation:

- Finds the first document where name is "zara."
- Deletes that document from the collection.
- If multiple "zara" exist, only the first one will be deleted.



deleteMany()

- Used to delete multiple documents that match a specific condition. All
- documents matching the filter will be removed.

- **db.collection.deleteMany({ <filter> })**

Ex : db.collection.deleteMany({ "age" : 23 })

- **Explanation:**

- Finds all students whose age is 20.**

- Deletes all those documents from the collection.**

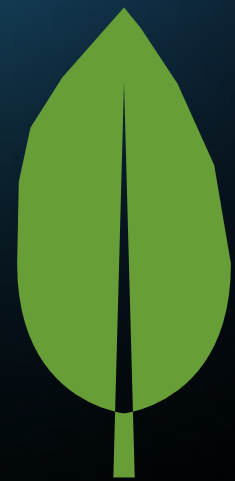


Delete All Documents

If you pass an empty filter {}, all documents in the collection will be deleted.

- **db.students.deleteMany({})**

✓ This removes every document from the students collection (but not the collection itself).

[Home](#)[About](#)[Content](#)[Others](#)

MONGODB QUERY FILTERS

(Comparison Operators)



What are Query Filters?

Query filters in MongoDB are conditions used inside the `find()` method to retrieve specific documents from a collection.

They help us fetch only the data that matches certain criteria.

```
db.students.find({ age: 20 })
```



MongoDB provides two main categories of query operators:

- 1. Comparison Operators**
- 2. Logical Operators**



1. Comparison Operators

Comparison operators allow you to match documents based on the value of a specific field

- ◆ **\$eq (Equal To)**

Used to match documents where a field is equal to a specified value.

```
db.students.find({ age: { $eq: 20 } })
```



Returns all students whose age is exactly 20.



- ◆ **\$ne (Not Equal To)**

Matches documents where the field's value is not equal to the given value.

```
db.students.find({ city: { $ne: "Delhi" } })
```

✓ Returns students whose city is not Delhi.

- ◆ **\$gt (Greater Than)**

Finds documents where the field's value is greater than the given value.

```
db.students.find({ marks: { $gt: 75 } })
```

✓ Returns students having marks greater than 75.



- ◆ **\$gte (Greater Than or Equal To)**

```
db.students.find({ age: { $gte: 22 } })
```

✓ Returns students whose age is 22 or older.

- ◆ **\$lt (Less Than)**

```
db.students.find({ marks: { $lt: 80 } })
```

✓ Returns students having marks less than 80.



- ◆ **\$lte (Less Than or Equal To)**

```
db.students.find({ marks: { $lte: 70 } })
```

✓ Returns students having marks 70 or below.

- ◆ **\$in (In Array)**

Matches documents where the field value exists in a given list.

```
db.students.find({ city: { $in: ["Delhi", "Mumbai"] } })
```

✓ Returns students who are from Delhi or Mumbai.



- ◆ **\$nin (Not in Array)**

Matches documents where the field value is not in the given list.

```
db.students.find({ city: { $nin: ["Delhi", "Mumbai"] } })
```



Returns students who are not from Delhi or Mumbai.



2. Logical Operators

Logical operators help you combine multiple conditions in a single query.

- ◆ **\$and**

All conditions inside \$and must be true for a document to match.

```
db.students.find({  
  $and: [  
    { marks: { $gt: 70 } },  
    { city: "Delhi" }  
  ]  
})
```



- ◆ **\$or**

At least one condition inside \$or must be true.

```
db.students.find({  
  $or: [  
    { marks: { $gt: 70 } },  
    { city: "Delhi" }  
  ]  
})
```

- ◆ **\$not**

Used to negate a condition (opposite match).

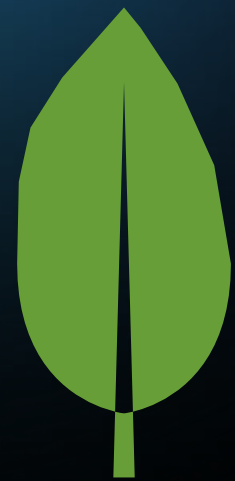
```
db.students.find({ marks: { $not: { $gte: 80 } }  
})
```

✓ Returns students whose marks are not greater than or equal to 80 (i.e., less than 80).



◆ **\$nor** Returns documents that don't match any of the given conditions.

```
db.students.find({  
  $nor: [  
    { city: "Delhi" },  
    { marks: { $gte: 80 } }  
  ]  
})
```



MONGODB QUERY FILTERS

(Combining Comparison and Logical Operators)



```
db.students.find(  
  { $and: [  
    { age: { $gte: 20 } },  
    { marks: { $lte: 80 } }  
  ]  
})
```

```
db.products.find(  
  $and: [  
    { price: { $gt: 30000 } },  
    { brand: { $in: ["HP", "Dell"]  
    } }  
  ]  
})
```



Conclusion

- MongoDB query filters are used to fetch only the documents you need. Comparison operators handle value-based filtering.
- Logical operators combine multiple conditions for complex queries.
- Together, they make MongoDB's querying system powerful and flexible.



ADVANCED QUERYING IN MONGODB



(**Sorting & Limiting Results**)



What is Advanced Querying?

Advanced querying in MongoDB means using special methods and operators to extract data more efficiently — such as sorting, limiting, skipping, and filtering results based on specific conditions.

In this section, we'll focus on sorting and limiting the output of our queries.



Sorting in MongoDB

Sorting means arranging the documents in a specific order — either ascending (1) or descending (-1).

```
db.collection.find().sort({ fieldName: 1 });
```

- 1 → Ascending order (A → Z or 0 → 9)
- -1 → Descending order (Z → A or 9 → 0)



Sorting in MongoDB

Sort by Price (Ascending)

```
>>> db.products.find().sort({ price:  
1 });
```

Sort by Price (Descending)

```
>>> db.products.find().sort({ price:  
-1 });
```

Sort by Multiple Fields

```
>>> db.products.find().sort({ category: 1, price:  
-1 });
```

✓ This first arranges by category alphabetically,
and within each category, sorts price in descending order.



Limiting Results

`limit()` is used to restrict the number of documents returned by the query

```
db.collection.find().limit(number);
```

```
db.products.find().limit(5);
```

This will show only the first 5 documents from the result set.



Combining Sort and Limit

You can combine both methods together — very useful when fetching top records, like top 5 most expensive products.

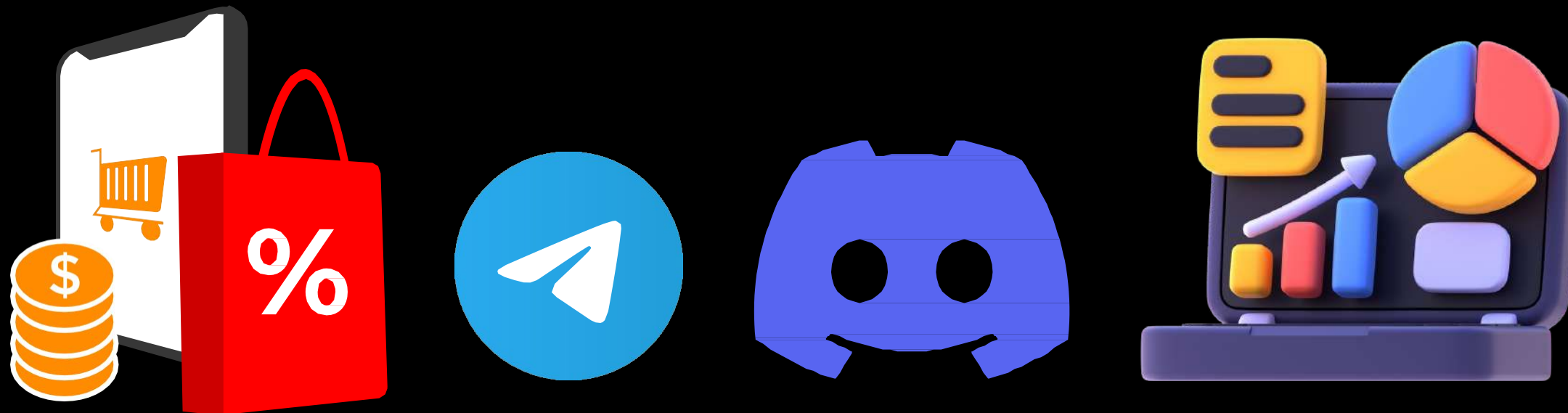
```
db.products.find().sort({ price: -1 }).limit(5);
```

✓ This fetches top 5 products with the highest price.



Real-Life Use Cases

- E-commerce site → Show Top 10 trending products
- Social media app → Fetch latest posts
- Analytics dashboard → Display top-performing users or sales





ADVANCED QUERYING IN MONGODB

( Skipping Results)

Pagination Concept

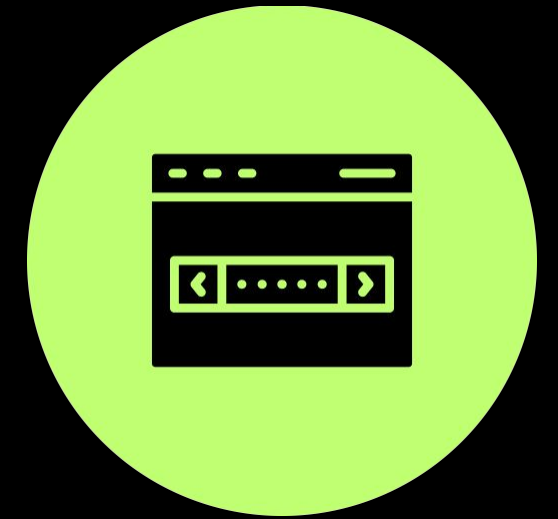


Skipping Results

What is skip()?

The skip() method is used to bypass a specific number of documents from the query result.

```
db.collection.find().skip(number);
```



It's commonly used in **pagination** — for example, when showing records page by page in an app.



Skipping Results

Skip the First 5 Records

```
db.products.find().skip(5)
```

👉 This will skip the first 5 documents and show results starting from the 6th document.

Combine Skip and Limit (Pagination)

```
db.products.find().skip(10).limit(5);
```

✅ This will skip the first 10 documents and then show the next 5 documents. Perfect for displaying page 3, page 4, etc. in a web app.



Sort + Skip + Limit (Complete Pagination)

```
db.products.find().sort({ price: 1 }).skip(10).limit(5);
```

👉 Sort by price (ascending), skip first 10, then show next 5 results.



Use Case:

- Page 1 →
 - .skip(0).limit(10) Page 2
 - → .skip(10).limit(10) Page
 - 3 → .skip(20).limit(10)
- and so on.



ADVANCED QUERYING IN MONGODB

(**Arrays in MongoDB**)





What is an Array in MongoDB?

In MongoDB, an array is a special data type that allows you to store multiple values in a single field.

💬 Think of it like a list — one field, but many items inside.

```
{  
  name: "Danish",  
  hobbies: ["coding", "reading", "cricket"]  
}
```

Here, hobbies is an array field, containing three values.



Why Use Arrays?

Arrays make your data more compact, organized, and realistic. Instead of creating multiple fields like hobby1, hobby2, hobby3, you can keep them all together inside one array.



Real-life use cases:

- A user's multiple hobbies
- Product tags in an e-commerce
- app A post's comments
- Order's list of products



Array

Data

```
db.users.insertMany([
  {
    name: "Raza",
    hobbies: ["coding", "cricket",
"reading"]
  },
  {
    name: "Tushar",
    hobbies: ["football", "traveling"]
  },
  {
    name: "vaishnavi",
    hobbies: ["reading", "painting",
"coding"]
  }
])
```



Find Documents Containing a Specific Value

```
db.users.find({ hobbies: "coding" });
```

- ✓ This finds all users who have “coding” in their hobbies array. MongoDB automatically checks inside the array elements.

Find Documents Where Array Contains Any of Several Values

```
db.users.find({ hobbies: { $in: ["reading", "traveling"] } });
```

- ✓ This returns users who have either reading or traveling as a hobby.



Find Documents That Contain All Specified Values — \$all

```
db.users.find({ hobbies: { $all: ["coding", "reading"] } });
```

- ✓ Finds users who have both coding and reading in their hobbies.

Find Documents Based on Array Size — \$size

```
db.users.find({ hobbies: { $size: 3 } });
```

- ✓ Returns all users who have exactly 3 hobbies.



Find Documents Where Any Array Element Matches a Condition

```
db.products.insertMany([
  { name: "Laptop", ratings: [4, 5, 3] },
  { name: "Phone", ratings: [2, 3, 1] },
  { name: "Tablet", ratings: [5, 4, 5] }
]);
```

```
db.products.find({ ratings: { $gt: 4 } });
```

- ✓ Returns documents where any rating is greater than 4.



Nesting objects in Arrays (Subdocuments)

Arrays of Objects

```
{  
  name: "chaitanya",  
  orders: [  
    { product: "Phone", price: 20000 },  
    { product: "Laptop", price: 60000 }  
  ]  
}
```

Here, orders is an array of objects.



Querying Arrays of Embedded Documents

Find Documents Based on Nested Field

You can use dot notation to query inside objects inside arrays 📍

```
db.users.find({ "orders.product": "Phone" });
```

✓ Returns all users who ordered a Phone.



ADVANCED QUERYING IN MONGODB

Projection in MongoDB (Selecting Specific Fields)



What is Projection in MongoDB?

When you query documents in MongoDB using `.find()`, it returns all fields by default.

But what if you only want specific fields — like just name and email? That's where Projection comes in!

Projection means —
Selecting which fields to include or exclude from the query result.



Show Only Selected Fields

-

```
db.users.find({}, { name: 1, email: 1 });
```

- ✓ Shows only name and email fields for all users.
- ✗ Other fields like `_id`, `age`, etc. are hidden.



Hide the `_id` Field

By default, MongoDB includes `_id` in every query.

```
db.users.find({}, { name: 1, email: 1, _id: 0 });
```

✓ Now `_id` will also be excluded.

Including vs Excluding Fields

In MongoDB:

1 → means include the field

0 → means exclude the field

```
db.users.find({}, { password: 0, age: 0 });
```



Projection with Conditions

You can combine filters and projections together.

```
db.users.find({ city: "Delhi" }, { name: 1, _id: 0 });
```

- ✓ Finds users from Delhi
- ✓ Shows only their names.



ADVANCED QUERYING IN MONGODB

(



Aggregation Pipeline



◆ What is Aggregation in MongoDB?

Aggregation in MongoDB is a powerful framework used to process data records and return computed results.

It's similar to SQL's GROUP BY and aggregate functions like SUM(), COUNT(), AVG(), etc.

Instead of simply fetching documents, aggregation transforms and analyzes them — allowing you to perform operations like:

- Filtering
- Groupin
- g
- Sorting
- Calculating totals/averages
- Reshaping data into new structures



◆ What is an Aggregation Pipeline?

An Aggregation Pipeline is a sequence of stages that process documents. Each stage takes the output of the previous stage and passes it to the next — just like a data processing pipeline.

Think of it like a factory assembly line:

Raw materials (documents) enter → go through processing → final product (transformed stages output) comes out.

MongoDB me Aggregation Pipeline ek data processing framework hai jisme data ko step-by-step stages me process kiya jaata hai — jaise ek pipeline me paani ek stage se dusri stage me flow karta hai.

Har stage me data transform (badla) hota hai, filter hota hai, group hota hai, calculate hota hai, ya sort hota hai.



Pipeline

Structure

The structure of an aggregation pipeline looks like this:

```
db.collection.aggregate([  
  { <stage1>: { <expression1> } },  
  { <stage2>: { <expression2> } },  
  ...  
])
```

Each stage performs a specific operation (like filtering, grouping, sorting, etc.).



Why Aggregation is Needed

In MongoDB when we have to work on large collections of data (like orders , users , products etc.),
And sometimes we have to generate reports and creates analytics from that data

Like :

- Total sales of each category
- Average rating per product
- Number of users per
- country Monthly income

report

To achieve these tasks we use aggregation so that we can summarize the data and get useful and meaningful information



In Aggregation We will cover

\$match

\$project → \$multiply, \$add, \$concat, \$toUpper, \$cond, etc.

\$group → \$sum, \$avg, \$max, \$min, \$push, \$first, \$last

\$sort, \$limit, \$skip

\$unwind → (for arrays)

\$lookup → (for joins)

\$facet → Parallel pipelines

\$bucket → Categorize numeric values

\$replaceRoot / \$replaceWith → Replace document structure

\$count → Count documents at any point

★ **\$match** filters documents based on specified conditions.

It is similar to the find() method.

Use

Select only required documents.

Filter data before performing expensive operations.

Reduce the number of documents passed to later stages.

Syntax : { \$match: { field: value } }

Example

Suppose we have:

```
{  
  name: "Raza",  
  age: 22,  
  city: "Pune",  
  marks: 85  
}
```

Query :

```
db.students.aggregate([ { $match: { city: "Pune" } } ])
```

Result : Only students whose city is "Pune" are returned.

★ \$project controls which fields should appear in the output and can also create new calculated fields.

Use

Include fields.

Exclude fields.

Rename fields.

Create calculated fields.

Syntax : { \$project: { field1: 1, field2: 1, field3: 0 } }

Note : 1 – Include , 0 – Exclude

Example

Suppose we have:

```
{  
  name: "Raza",  
  age: 22,  
  city: "Pune",  
  marks: 85 }
```

Query :

```
db.students.aggregate([  
  { $project: { name: 1, marks: 1, _id: 0 } }  
])
```

Result : {
 name: "Raza",
 marks: 85
}

1) \$multiply multiplies two or more numeric values.

Use

Used for calculations such as:

Price × quantity

Salary calculations

Marks calculations

Tax calculations

Syntax : { \$multiply: [value1, value2, ...] }

Example

```
db.products.aggregate([ { $project: {  
  name: 1,  
  totalPrice: { $multiply: ["$price", "$quantity"] }  
} } ])
```

If :

price = 500

quantity = 3

Output : totalPrice: 1500

2) \$add adds two or more numeric values.

Use

Adding marks
Calculating totals
Adding tax
Date calculations

Syntax { \$add: [value1, value2, ...] }

Example

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      totalMarks: {
        $add: ["$maths", "$science", "$english"]
      }
    }
  }
])
```

If :

Maths = 80, Science = 75, English = 85

Then :

totalMarks = 240

3) \$concat joins multiple strings into a single string.

Use

Creating full names.

Creating addresses.

Syntax { \$concat: ["\$field1", " ", "\$field2"] }

Example

```
{ firstName: "Mohammad", lastName: "Raza" }
```

Query :

```
db.students.aggregate([ { $project:
  { fullName:
    { $concat: [ "$firstName", " ", "$lastName" ] } }
  } ])
```

Output :

```
{ fullName : Mohammad Raza }
```

Some more \$project functions :

\$toUpper converts a string into uppercase letters

\$toLower converts a string into lowercase letters.

\$cond performs a conditional operation, similar to an if-else statement.

- ☞ **\$group** : used to groups documents based on a specified field and performs calculations on each group.

Used for:

Counting documents.

Calculating totals.

Calculating averages.

Finding maximum/minimum values.

Syntax : { \$group: { _id: "\$field", result: { accumulator: "\$field" } } }

Example : **db.students.aggregate([{ \$group: { _id: "\$city" } }])**

Output : { _id: "Pune" } { _id: "Mumbai" } { _id: "Nashik" }

Note : _id defines the grouping key.



Common \$group Accumulators :

Function	Meaning / Use	Example
\$sum	Calculates total or counts documents	{ \$sum: "\$marks" }
\$avg	Calculates average	{ \$avg: "\$marks" }
\$max	Finds highest value	{ \$max: "\$marks" }
\$min	Finds lowest value	{ \$min: "\$marks" }
\$push	Creates an array containing values from all documents	{ \$push: "\$name" }
\$first	Returns the first value in the group	{ \$first: "\$name" }
\$last	Returns the last value in the group	{ \$last: "\$name" }



Example

```
students.aggregate([ {  
  $group: { _id: "$city",  
  totalMarks: { $sum: "$marks" },  
  averageMarks: { $avg: "$marks" },  
  highestMarks: { $max: "$marks" },  
  lowestMarks: { $min: "$marks" },  
  students: { $push: "$name" } } } ])
```

\$group

—	\$sum	→ Total / Count
—	\$avg	→ Average
—	\$max	→ Highest
—	\$min	→ Lowest
—	\$push	→ Array of values
—	\$first	→ First value
—	\$last	→ Last value

Key point: `_id` defines what you want to group by, while the accumulator functions define what you want to calculate from each group.

🌀 \$lookup in MongoDB Aggregation Pipeline

◆ What is \$lookup?

\$lookup is a MongoDB aggregation stage used to join documents from two collections, similar to how SQL JOIN works.

It allows you to combine data from different collections based on a common field (a key that relates them).

◆ Purpose

The main goal of \$lookup is to fetch related data from another collection without having to make multiple queries.

It's especially useful when your data is normalized (spread across multiple collections).



◆ **Basic Concept**

- \$lookup performs a left outer join between two collections.
- It adds a new field (usually an array) that contains the matching documents from the “joined” collection.
- If no match is found, the field will contain an empty array.

◆ **Key Fields of \$lookup**

- from – Name of the collection to join with.
- localField – Field from the input (source)
- collection.
- foreignField – Field from the “from” collection to match against.
- as – The name of the new array field that will store



Example



users Collection

```
[
  { "_id": 1, "name": "Danish", "city": "Delhi" },
  { "_id": 2, "name": "Aliza", "city": "Mumbai" },
  { "_id": 3, "name": "Ahmed", "city": "Kolkata" }
]
```



orders Collection

```
[
  { "_id": 101, "product": "Laptop", "price": 50000, "userId": 1 },
  { "_id": 102, "product": "Phone", "price": 20000, "userId": 1 },
  { "_id": 103, "product": "Book", "price": 500, "userId": 2 }
]
```



Aggregation Query

```
db.users.aggregate([
  { $lookup: {
    from: "orders", // target collection
    localField: "_id", // field in users
    foreignField: "userId", // field in orders collection
    as: "userOrders" // output array field name
  }
}]
```



1. What are Indexes & Why use them?

Indexes in MongoDB are like the index of a book. Instead of reading every page to find a topic, you look at the index and directly jump to the right page.

Similarly, MongoDB uses indexes to find documents faster without scanning the entire collection

Without Index:

MongoDB performs a Collection Scan (COLLSCAN) — it checks every single document in the collection to find matches.

👉 This is slow, especially when you have thousands or millions of records.



With Index:

MongoDB performs an Index Scan (IXSCAN) — it directly looks up data through the index, similar to how a dictionary helps you find a word quickly.

Why Use Indexes:

- ⚡ Increases query performance
- 🚀 Reduces execution time
- 🧠 Optimizes search, sort, and filter operations
- 💾 Useful for large datasets

Example use cases:

- Searching by a username or email in a user database
- Sorting blog posts by date
- Filtering orders by customer ID



Creating Indexes (createIndex)

To create an index in MongoDB, you use the createIndex()

method. Syntax:

```
db.collection.createIndex({ fieldName: 1  
})
```

- **1** → Ascending order
- **-1** → Descending order



Example

```
db.users.createIndex({ username: 1 })
```

This creates an index on the username field. Now, whenever you query like:

```
db.users.find({ username: "danish" })
```

MongoDB will use the index to find results super fast ⚡.

Pro Tip :

You can also create multiple indexes for different fields, but avoid creating too many — they consume extra storage and slow down writes.



MongoDB

Optimizing

Compound Indexes & Text Indexes



👉 Compound Indexes

A compound index includes multiple fields in one index. Useful when queries filter or sort on multiple fields.

```
db.users.createIndex({ firstName: 1, lastName: 1 })
```

Now if you query:

```
db.users.find({ firstName: "Coding", lastName:  
"Hunger" })
```

MongoDB uses this compound index for better performance.



Important Note:

The order of fields in compound indexes matters!

Example: { firstName: 1, lastName: 1 } can help with queries

- using: both fields 
- only firstName 
- only lastName 

So, plan your indexes according to query pattern



Text Indexes

Text indexes allow you to perform text search on string content — like searching inside titles, descriptions, or articles.

So instead of doing exact matches like:

```
db.articles.find({ title: "MongoDB  
Tutorial" })
```

You can search for any text that “contains” or “relates” to MongoDB, like:

```
db.articles.find({ $text: { $search: "mongodb" } })
```

This helps MongoDB behave like a mini search engine, just like Google, but inside your database.



Why We Need Text

Indexes

```
db.articles.find({ title: /mongodb/i })
```

This uses regular expressions, which scan every document — super slow 🚫
for large collections.

But if you use a text index, MongoDB internally creates an inverted index
(like a dictionary) of all the words and their positions.

This makes text searches much faster and smarter.

✓ Without text index:

Scans all documents → Slow

✓ With text index:

Uses inverted word lookup → Lightning fast ⚡



How to Create a Text Index

You use the `createIndex()` method just like normal indexes, but instead of 1 or -1, you use "text".

```
db.articles.createIndex({ content: "text" })
```

Now, MongoDB will analyze every document's content field, split it into words (tokens), and store them in an internal index

[Home](#)[About](#)[Content](#)[Others](#)

MongoDB

Caching



What is Caching?

Caching means storing frequently accessed data in memory (RAM) so that it can be retrieved quickly, without going all the way to the disk every time.

In simple words —

“Cache is like a short-term memory for your database.”

When your application asks MongoDB for some data, instead of reading it from the slower disk each time, MongoDB can serve it from its in-memory cache, making queries much faster.



2. Why Caching is Important

Databases are often used by high-traffic applications — websites, dashboards, APIs, etc.

If MongoDB has to read from disk every time, it becomes slow.

Caching helps to:

- ⚡ Speed up queries (especially for frequently used data)
- 🧭 Reduce disk I/O (fewer reads from storage)
- 💻 Handle more concurrent users efficiently
- 🧮 Lower CPU and disk usage
- ↺ Improve overall system performance

So, caching acts as a performance booster for MongoDB

3. How MongoDB Handles Caching

Internally, MongoDB automatically manages caching using its WiredTiger Storage Engine (which is the default engine).

You don't have to write extra code for it — MongoDB takes care of caching behind the scenes.

Let's break it down:

a) **WiredTiger Cache**

- WiredTiger uses an internal cache (in-memory) to store recently accessed documents, indexes, and metadata.
- When a query runs, MongoDB first checks if the required data is already in cache.
 - If yes → served instantly from memory (super fast 🚀)
 - If no → fetched from disk and then stored in cache for future use.

This mechanism follows a "Least Recently Used (LRU)" pattern — old or unused data is gradually removed to make space for new data.



3. How MongoDB Handles Caching Internally

b) Cache Size

- By default, MongoDB allocates 50% of the system's RAM for the WiredTiger cache (on dedicated servers).
- For example, if your server has 16 GB RAM → MongoDB will use about 8 GB for caching.

You can tune this limit if needed, but MongoDB usually manages it efficiently.

[Home](#)[About](#)[Content](#)[Others](#)

THANK YOU

Mohammad Raza